

Transcript

Module 7: Frontend Frameworks

Video 1: Overview

Welcome to the Frontend Framework module. This module is designed to equip you with a foundation to appreciate modern frontend development, particularly focusing on frontend frameworks. By the end of this course, you will be equipped with the foundational knowledge of frontend frameworks. This knowledge will be useful to allow you to appreciate why frontend frameworks exist and better prepare you before you learn the actual coding. The course is divided into several topics: Introduction to Frontend Frameworks, Traditional vs. Modern Development Approaches, Components-based Design, Declarative Approach to Development, Single Page Applications and Getting Started with Frontend Development Frameworks.

For the first topic on Introduction to Frontend Frameworks, we'll explore the basics of frontend frameworks, why they are essential, and the advantages they offer over traditional methods.

We will cover popular frameworks like React, Angular and Vue.js.

Following that, we will compare the traditional and modern approaches to frontend development. You will learn about the evolution of web development practices, the limitations of older methods and how modern techniques addresses these challenges.

Components-based design is foundational to modern frameworks. We will discuss the principles of components-based architecture, how to build reusable components and what are the best practices for component design.

Declarative programming is another key concept in modern frontend development. In this part of the course, we will delve into how declarative syntax simplifies development, improves code readability and maintainability. When developing with modern frontend frameworks, developers can design their web application to be a single page application. We will discuss what Single Page Applications are and how they provide a seamless user experience by loading content dynamically.

In the final part of this module, we will briefly highlight how to get started with frontend development using modern frameworks. You will learn about tools and libraries that are used in development. We will also walk through a basic project set-up.

This concludes our module overview. Thank you for attending this module, and I'm excited to start this journey with you!

Video 2: Introduction to Frontend Frameworks

Welcome to the lecture on 'Introduction to Frontend Frameworks'. Here are the learning objectives. By the end of this lecture, you should understand the role and importance of frontend frameworks in modern web development and be able to identify key features and benefits of using frontend frameworks.

Frontend frameworks are software frameworks specifically designed for building user interfaces. They provide structure and tools that facilitate efficient development. Some of the popular examples include React, Angular and Vue.js. Each of these frameworks offer unique features and benefits but generally they all have the same objective.

The first major reason to use frontend frameworks is efficiency. These frameworks speed up development by providing reusable components. Instead of writing code from scratch for every part of your application, you can leverage pre-built components that save significant time and effort. The second reason is consistency. Frontend frameworks enforce a standardised code structure, ensuring that your code base remains organised and uniform. With regards to UI and UX, this standardisation usually also results in a consistent look and feel across your application, which is crucial for user experience. The third reason is maintenance. Frontend frameworks make it easy to update and maintain your applications. Their structured approach and reusable components facilitate better scalability, allowing your application to grow while still keeping it manageable.

One of the key features of frontend frameworks is component-based architecture. This means that your UI elements are encapsulated into reusable components, which enhances code reusability and maintainability. Each component can be developed and tested independently from each other. Another key feature is state management. Efficient data handling and predictable state changes are essential for building dynamic, responsive applications. Frontend frameworks usually provide a way to manage the state of your application in a systematic manner. Routing capabilities are another crucial feature. They enable single-page application navigation, which improves user experience by allowing smoother transitions between different parts of your application without reloading the entire page.

Additionally, data binding is another important aspect, facilitating automated synchronisation of data between model and the view. Tooling is an essential aspect of frontend frameworks. Tools like Webpack and Babel are used for building and optimising your application. Features like code splitting and lazy loading enhance performance by loading only the necessary parts of your application.

Using frontend frameworks significantly improves productivity. With pre-built components and the ability to reuse code, developers can build applications faster and more efficiently. Frontend frameworks also enhance user experience. Faster load times and smooth navigation are achieved through efficient rendering and routing capabilities, leading to a more seamless and enjoyable user experience. Another benefit is the strong community support. Frontend frameworks often have extensive documentation, active forums and numerous tutorials. This support network can be invaluable when learning the framework or troubleshooting issues.

Now, let's take a look at React. React is known for its virtual DOM, which optimises rendering performance. It follows a components-based architecture and uses a JSX syntax, which allows HTML to be written within JavaScript. Here is a simple React component. This example demonstrates how easy it is to create a component in React. It was originally created by Facebook, now Meta.

Next, we have Angular. Angular features two-way data binding, dependency injection and strong TypeScript support. It is a full-featured framework developed by Google. Here is a simple Angular component. This component shows how to define a component using React Component Decorator.

Lastly, let's briefly discuss Vue.js. Vue.js offers reactive data binding, a simple and flexible API, and a command line interface for scaffolding projects. It is an independent, progressive framework known for its ease of learning. Here is a simple Vue.js component. This example illustrates how to create a Reactive Vue instance.

React is a JavaScript library that allows for fast rendering. It has a large community and is probably the most popular among the three frameworks as of the recording of this video. Angular, on the other hand, is a full-featured framework and has also a strong community support and is used in various enterprise solutions.

Vue.js being an independent project is lightweight, easy to learn and has great documentation. Each framework has its own strength and you probably will not go wrong with any of these frameworks.

In general, frontend frameworks are particularly useful for large-scale applications development where you need to develop complex UI. They allow developers to produce single-page applications and allow projects to be built in an efficient and effective manner. Here are the factors that you should consider when choosing a framework.

The project requirements, whether it is an enhancement through an existing system or you are building a system from scratch. Since there are some learning curves, and different projects might have different timeline constraints, it is important to consider the team expertise before deciding on the framework. If there are specific performance needs, it is good to investigate further whether the framework fits your performance requirements. Lastly, it is important to consider the community support and ecosystem.

React, Angular and Vue.js are popular frameworks that are well discussed online. However, there might be new frameworks that are released by the time that you are watching this video. It is a good idea to ensure that there are good documentations and community support and interest since the lifecycle of a system goes beyond the initial development process.

In summary, we have covered the role and importance of frontend frameworks in modern web development. We have also covered the key features and benefits of using frontend frameworks.

Video 3: Traditional Approach to Frontend Development

Welcome to the lecture on 'Traditional Approach to Frontend Development'. Here are the learning objectives. By the end of this lecture, you should have a solid understanding of what traditional approach to frontend development is, and the common issues associated with this approach.

HTML, CSS and JavaScript are used in frontend development, in both the traditional and the modern approach. However, the traditional approach focuses heavily on server-side rendering, where the server is responsible for generating the HTML content and sending to the client. Unlike modern frontend frameworks, this method does not leverage client-side rendering capabilities.

Let's recap the basic components of web development. HTML defines the structure and content of web pages. It acts as a skeleton of a webpage. CSS is responsible for the styling of web pages. It defines how HTML elements are presented on the screen. JavaScript adds interactivity to web pages. It allows us to manipulate the DOM and respond to user actions dynamically.

Server-side rendering or SSR involve generating HTML on the server and sending the entire page to the client. Some examples of SSR technology include PHP and ASP.NET. Since the HTML of the page is generated on the server site, the web browser can easily display the web page with minimal client-side computation.

Just to reiterate what we have covered, the traditional approach is characterised by synchronous loading of pages where each page is loaded entirely on the server. It is the server's responsibility to render the page so the client can be thought off as a thin client, which does very little computation. In the situation where the pages need additional DOM manipulation, developers usually will write additional JavaScript to manually update the DOM on specific events. One of the issue with such approach is maintainability. Updating large code base can be cumbersome. There is often a tight coupling between backend and frontend technologies, making it hard to separate the concern effectively.

Backend and frontend developers need to work very closely together, which also slows down development process. Another significant problem is reusability. Code is often written for specific pages, making it very difficult to reuse components across different parts of the application. Developers may end up copying codes to achieve similar UI effects, leading to duplication and inconsistency. Performance is also another concern. The server generates the entire page for every request, causing full page reloads when navigating across pages. This increases server load and latency, resulting in higher data usage for both the user and the server. The user experience can suffer as well.

Full page reloads are disruptive and can result in poor responsiveness. For example, clicking on a link reloads the entire page, causing the user to lose the scroll position. This can be frustrating, leading to less engaging experience.

Lastly, scalability is a challenge. As the application grows in complexity, it becomes harder to manage. Performance bottlenecks can occur when handling increased user load and can be very expensive. Traditional methods struggle to keep up with the demands of modern web applications.

In conclusion, while the traditional approach to frontend development laid the foundation for web development, it has several limitations in terms of maintainability, reusability, performance, user experience and scalability. Understanding these issues sets the stage for exploring modern frontend frameworks, which address many of these challenges and offer more efficient ways to build robust web applications.

Video 4: Modern Approach to Frontend Development

Welcome to the lecture on 'Modern Approach to Frontend Development'. Here are the learning objectives. By the end of this lecture, you should have a clear understanding of what modern approach to frontend development entails, the rationale behind its adoption and the benefits it offers. Modern frontend development revolves around the use of JavaScript frameworks and libraries such as React, Angular and Vue.js. This approach primarily focuses on client-side rendering and emphasises on components-based architecture, enabling a more dynamic and responsive web application.

Client-side rendering is a method where the initial load fetches the entire web application, typically written in JavaScript. The data to be displayed for the page is usually fetched separately through web services. Instead of loading the entire HTML for each page, JavaScript renders and generates the UI programmatically, leading to reduced server load and latency.

Client-side rendering brings several advantages. There is a reduced overall data usage on both the server and client, leading to lower cloud hosting expenses and reduced mobile data usage for users. Lower data transfer also results in shorter waiting time. Ultimately, enhances the user experience significantly.

However, client-side rendering also has several drawbacks. There is a longer waiting time for the first load to the website because the JavaScript powering the entire site must be first downloaded. Client devices need to be capable to handle JavaScript processing, but this is usually not an issue anymore as modern browsers can support execution of modern JavaScript syntax. Furthermore, client devices need to do processing, which means that they need to be relatively powerful devices. On the positive side, this shifts some of the computational load from the server to the client, potentially lowering the server costs.

The key characteristics of modern frontend development include components-based design, reactive and declarative programming, single page application, and the use of built tools and module bundlers.

Components-based design involves creating reusable UI components that encapsulate their own HTML, CSS and JavaScript.

For example, a button component can have its own style and behaviour, making it easier to reuse across different parts of the application. The declarative approach to development allows developers to specify the intended behaviour of the application, such as binding the UI with the data. The framework then takes care of the updating of the UI when the data changes. Here is an example of the React State and Prop, which allows us to achieve a declarative programming approach.

Single page applications load a single HTML page and dynamically update the content without requiring full page reloads. This leads to a more seamless and enhanced user experience. Built tools automate various development tasks such as code transpilation and minification. Module bundlers like Webpack package code efficiently, ensuring that applications load quickly and run smoothly.

The rationale for adopting a modern approach to frontend development includes the following. Improved user experience where there's a seamless interaction without full page reloads. Better performance since there are reduced server load and faster page transitions. Enhanced development workflow which includes developing modular codes and reusable components. The benefit of modern frontend development are better modularity, maintainability and scalability.

In conclusion, modern frontend development focuses on improved user experience, performance and developer productivity. By embracing components-based architecture, single page application and client side rendering, it can allow us to build a more maintainable and scalable applications.

Video 5: Component based design

Welcome to the lecture on 'Components-Based Design'. Here are the learning objectives. By the end of this lecture, you should have a clear understanding of how to organise web applications into components. The core idea behind components-based design is to organise web applications into reusable, self-contained components. These components serve as the basic building blocks to create a user interface. In essence, the entire user interface composed of these components, and a single component can contain other components within it. Throughout this lecture, we will mainly be using React to illustrate this concept and show you how this approach is implemented in practice.

Consider the following page as an example. The entire page is treated as one large component. Now within the main component, there are other components: The header and the body and the footer. Each of these sections is a component in itself, contributing to the overall structure of the page. This hierarchical organisation makes it easy to manage different parts of the application, ensuring that each part can be developed independently and maintained independently. Drilling down further, the body components can contain a sidebar and bodycontent. This illustrates how components can nest within each other. The sidebar can contain menubar, which in itself can contain menuitems.

In React, components extend the HTML vocabulary. This means that you can write HTML within our JavaScript code, using a syntax known as JSX. JSX allows us to create components that are both powerful and easy to understand, blending HTML and JavaScript in a way that enhances the development process. Here is an example of a React component. Notice how we write HTML within our JavaScript. This is JSX, which allows React to interpret and render HTML elements. This approach allows us to perform any necessary logic before rendering the HTML, including defining nested functions within the component function. This flexibility is one of the reasons why React is such a powerful tool for building web applications.

When organising components, it's crucial to structure them into a tree-like hierarchy. Components should have a clear parent-child relationship. We should divide the responsibility across different components to adhere to the principle of separation of concern. This approach ensures that each component is focused on a single task, making the overall application easier to manage and maintain. Here are some best practices to consider when designing components.

Keep components small and focus on a single task. We use components wherever possible to avoid unnecessary redundancy. Maintain a consistent naming convention to keep your codebase organised and readable. By adhering to these best practices, we can create applications that are not only robust but also easy to maintain and scale.

In summary, components-based design significantly improves the maintainability, scalability and readability of web applications. Organising web applications into components is essential for modern frontend development. This approach not only makes development more efficient but also results in a better-structured and more manageable codebase. As we continue exploring frontend frameworks, keep these principles in mind, as they will be fundamental to the success of your journey in building modern web applications.

Video 6: Declarative Approach to Development

Welcome to the lecture on 'Declarative Approach to Development'. Here are the learning objectives. By the end of this lecture, you should have a solid understanding of what declarative approach to development entails and appreciate its benefit in web development. Traditionally, developers are responsible for handling the rendering of the page. This means directly manipulating the document object model, or DOM. Whenever there is a change in the data, developers had to manually update the DOM to reflect these changes. This process could become quite cumbersome and error-prone, especially as the application grows in size and complexity.

Let's take a look at a simple example over here of DOM manipulation. Here, we have a HTML button with an onClick attribute that calls a JavaScript function addItem. Below it, there's an unordered list. When a button is clicked, a new item is added to this list. The JavaScript code that powers this behaviour is shown here. The addItem function appends a new item onto the list each time when the function is triggered. While this approach works, it requires the developers to manage the state and update the DOM manually.

Now, let's consider the challenges that arise from such approach. React was created by Facebook. What do you think were some of the challenges that Facebook developers faced? There are many features on Facebook web portal and various events can affect the user interface. These events include updating of the wall feed, post receiving comments, receiving an instant message from friends, and so on and so forth. There are many features on the Facebook web portal, and various events that can affect the user interface. This event includes updating of the wall feed, post receiving comments, receiving an instant message from a friend.

Imagine the different development team will now have to work closely with each other to ensure that all the necessary UI updates are rendered quietly on the web portal. Furthermore, to add to the difficulty, some of these UI updates can happen at the same time. So each UI update does not just involve UI manipulation for just their own part of the web page. This problem is not just specific to Facebook. As a web application grows, managing complex DOM manipulation will become increasingly cumbersome and error-prone. This is also the main reason that gave rise to the creation of React, which aims to simplify the process by extracting away the direct DOM manipulation.

With modern frontend frameworks, the burden of rendering and re-rendering of the UI is now shifted from the developers to the framework. Developers only need to declare what they need, such as binding the UI to the data. When the data changes, the framework will automatically update the UI accordingly, ensuring consistency and efficiency. The declarative approach simplifies frontend development by allowing developers to define the desired state of UI, and the framework will take care of the rendering. This means we describe what the UI should look like given certain data, and the framework will handle the actual update. This abstraction significantly reduces the complexity of managing UI state.

For a large-scale application, this approach is a game-changer. It allows a more systematic way to develop frontend applications. By delegating the rendering responsibility to the framework, developers can focus on the core business logic, improving efficiency and reducing the likelihood of error. For instance, updating a list of multiple items becomes more efficient as the framework would handle the batch edit as a single update rather than the developer having to manage each of the updates individually.

To summarise, we have explored the concept of declarative approach to development and its benefits in the context of modern frontend frameworks. By understanding and leveraging this approach, you can create more efficient and maintainable and scalable web applications. This fundamental shift of development practice is at the heart of why frameworks like React, Angular, Vue.js are so powerful and widely adopted.

Video 7: Single Page Applications

Welcome to the lecture on 'Single Page Applications'. Here are the learning objectives. By the end of this lecture, you should have a clear understanding of what single page applications are. We will explore their structure, benefits and challenges, and see how they differ from traditional web applications. So, what exactly is single page application, or SPA? It is a web application that loads a single HTML page and dynamically updates the content as the user interacts within the application. The key characteristics of a single page application is that it does not require a full-page reload to change views or perform actions.

In traditional web applications, each view typically resides in its own separate physical page. When a user navigates to a different part of the application, a new page is loaded from the server, and the browser will perform a full-page reload. This means the user will have to wait for the new page to load entirely. While both traditional web applications and single page applications have the concept of different views and pages, the difference lies in their implementation. In a traditional multi-page application, different pages mean different HTML files. In SPA, however, there is only one physical HTML file. The illusion of multiple pages is achieved by dynamically swapping the view of the single page during runtime.

Let's delve into how single page applications function. A SPA typically consists of a single HTML page. JavaScript plays a crucial part by handling routing and view updates. When data needs to be fetched from the server, it's done asynchronously through technologies such as AJAX or the Fetch API. This enables navigation within the application without triggering a full-page reload.

To illustrate this, consider Gmail. When you access your inbox, the URL looks something like the following. Now, when you switch to the sent folder, the URL changes from the inbox section to the sent section. In terms of the URL, this technically is the same main URL, even though the user gets the illusion that there's two separate pages. One of the significant benefits of SPA becomes evident in situations when there is network issues.

Suppose there is a network disruption and your machine loses its Wi-Fi connection. If you navigate back to the inbox page, the content is still available for the user because the data is loaded previously. This offline capability enhances the user experience significantly.

Apart from the ability to handle network downtime, single page applications offer improved user experience due to faster interactions, as they reduce the number of server requests needed for the navigation. For Transition between views are smoother, providing a more responsive and engaging interface for users. Some of the key benefits of SPA includes asynchronous content loading, dynamic updating of the user interface and client-side routing. This feature works together to provide seamless user experience.

However, SPAs are not without their challenges. They often have a longer initial loading time since the entire application needs to be loaded up front. SEO consideration can be tricky because search engines may have difficulty indexing content that are dynamically loaded. Browser compatibility and complexity of managing the application state can also pose a significant challenges for older web browsers.

In summary, we covered what single page applications are, how they work, their benefits and challenges associated with them. SPAs represent a modern approach to web development that enhances user experiences through dynamic and seamless interaction.

Video 8: Getting Started with Frontend Development Framework

Welcome to the lecture on 'Getting Started with Frontend Development Frameworks'. Here are the learning objectives. By the end of this lecture, you should have an understanding of how to get started with frontend development without any software installation and the typical software installation that is needed for frontend development. The easiest way to get started with frontend development framework is to use a cloud development environment. These platforms allow you to dive into coding without having to worry about setting up your local environment. Two excellent platforms for this purpose are as follows: CodeSandbox and StackBlitz.

The primary benefit of using a cloud-based development environment is that there is no setting up required. Setting up the environment on your local machine can sometimes be time-consuming and prone to issues. By using a cloud-based environment, you can focus on learning how to code rather than dealing with the setting up issues. This approach simplifies the initial learning curve and lets you get started with coding faster.

Let's walk through an example of using CodeSandbox. First, head over to the CodeSandbox website and sign in. If you don't already have an account, you can create one. Once you are signed in, you can create a new sandbox. This sandbox will be your playground where you can write and test your codes. You can specify the type of app that you are developing. For example, if you are creating a React app, you can specify React for the sandbox. This will set up a basic React environment for you. If everything is set up correctly, you should see a web-based code editor with a preview panel on the right. This panel shows how your application looks in real-time as you write your codes.

Now, let's talk about the typical software installation you might need if you choose to set up your local development environment First, you will need to install Node.js from the official Node.js website. Node.js is a JavaScript runtime environment that allows you to run JavaScript on your machine such as a server. Along with Node.js, you will need the Node Package Manager or NPM. It is a package manager for JavaScript. It usually comes pre-bundled with the Node.js installation. Installing packages using NPM helps you manage dependency in your projects more easily.

For coding, a popular code editor such as Visual Studio Code is preferred. You can download it from the VS Code website. It offers a range of features and extensions that makes your coding experience more efficient and enjoyable.

To summarise, we have covered two key topics. First, we explored how to get started with front-end development using a cloud-based environment, which requires no setting up and allows you to focus on coding. Second, we discussed the typical software installation needed to set up a local environment, including Node.js, NPM and Visual Studio Code.

Video 9: Module Summary

Welcome to the module summary lecture on 'Frontend Frameworks'. In this video, we'll recap the key topics we have covered, which include: Introduction to Frontend Frameworks, Traditional Approach to Frontend Frameworks, Modern Approach to Frontend Development, Component-Based Design, Declarative Approach to Development, Single Page Applications and Getting Started with Frontend Development Frameworks. This summary will provide a quick overview of the main points from each topic.

Frontend frameworks like React, Angular and Vue.js are essential tools for modern web development. They offer efficiency through reusable components, ensuring consistency with standardised code structures and facilitated maintenance with their structured approach. Key features include components-based architecture, state management and routing capabilities. Using these frameworks significantly improves productivity and improves user experience through faster loading time and smoother navigation.

The traditional approach to frontend development relies on server-side rendering, where the HTML is generated on the server and sent to the client. This method often results in maintainability issues, lack of code reusability, performance bottlenecks, poor user experience due to full page reload and scalability challenges. Despite laying the groundwork for web development, it struggles to meet the demands of modern applications.

Modern frontend development focuses on client-side rendering, utilising JavaScript frameworks and libraries to create dynamic and responsive applications. Key characteristics include component-based design, declarative programming, single page application, and the use of build tools like Webpack and Babel. This approach improves user experience, performance and development workflow by promoting modularity, maintainability and scalability.

Components-based design organised web applications into reusable, self-contained components. These components encapsulate their own HTML, CSS and JavaScript, enhancing code reusability and maintainability. Using a tree-like hierarchy, components are structured in a clear parent-child relationship, adhering to a principle of separation of concern. This approach significantly improves maintainability, scalability and readability of web applications.

The declarative approach shifts the burden of rendering and updating the UI from the developer to the framework. Developers declare the desired state of the UI, and the framework handles the extra updating on the DOM. This simplifies the development process, reducing the complexity of managing UI state and improving efficiency. By leveraging this approach, developers can better focus on developing the business logic, at the same time creating more efficient and maintainable and scalable web applications.

Single page applications load a single page and dynamically update the content without a full-page reload. This enhances user experience by providing faster interactions and smoother transition between views. Single page applications use asynchronous content loading and client-side routing, allowing for a more responsive and engaging interface. We have seen the example from Gmail that allows for seamless navigation and offline capability.

To get started with frontend development frameworks, choose a framework that best fits your project requirements, team expertise and performance needs. Consider the community support and ecosystem for the framework. Popular frameworks like React, Angular and Vue.js offer extensive documentation and active community, making it easier to learn and troubleshoot issues.

In summary, we have covered the role and importance of frontend frameworks, traditional versus modern approaches to front end development, components-based design, the declarative approach of development, single page application and getting started with frontend frameworks. These concepts and tools are essential for building efficient, maintainable and scalable web application. Thank you for your attention. I hope that this module has provided you with valuable insights into frontend development.

THE END